Log-based Anomaly Detection with Transformers Pre-trained on Large-scale Unlabeled Data

Senming Yan[1,2], Lei Shi[3], Jing Ren[4,5], Wei Wang*[4], Yaxin Liu[1,2], Limin Sun[1,2], Xiong Wang[5], Wei Zhang[6]
[1] Beijing Key Laboratory of IOT Information Security Technology, Institute of Information Engineering, CAS, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3] School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China
[4] Department of Strategic and Advanced Interdisciplinary Research, Peng Cheng Laboratory, Shenzhen, China
[5] University of Electronic Science and Technology of China, Chengdu, China
[6] School of Electrical Engineering and Telecommunications, University of New South Wales, Sydney, Australia
Dongguan University of Technology, Dongguan, China
Email: {yansenming, liuyaxin0232, sunlimin}@iie.ac.cn, shi.lei@sjtu.edu.cn,
{renjing, wangxiong}@uestc.edu.cn, wangw01@pcl.ac.cn, w.zhang@unsw.edu.au

ICC 2024 - IEEE International Conference on Communications | 978-1-7281-9054-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICC51166.2024.10623067

***Abstract*—It is crucial to automatically detect anomalous patterns in system logs to protect computer systems from cyber attacks and malfunctions. However, as log data is becoming increasingly complex and labeled logs are difficult to obtain, it poses serious challenges to existing methods. To this end, this paper introduces the pre-training and fine-tuning paradigm to the log analysis domain and proposes a novel log anomaly detection framework. We propose the masked log reconstruction approach to pre-train a Transformer-based foundation model and fine-tune it for the event prediction task to obtain the anomaly detector. Our training methods exploit the sequential information within unlabeled logs with self-supervised learning. Experimental results on two public datasets demonstrate the performance superiority of our framework compared with existing state-of-the-art methods. More importantly, it is suitable for real-world scenarios where labeled logs are difficult to acquire.**

***Index Terms*—Cyber security, log analysis, log anomaly detection, pre-trained model**

I. Introduction

The maintenance of normal operations and high availability in computer systems is imperative, as internal errors and external cyber attacks can precipitate decreases in availability, resulting in substantial security vulnerabilities and financial losses. Modern computer systems generate logs to record critical runtime information [1]. The ever-increasing complexity of computer systems and the exponentially growing amount of logs render manually analyzing logs infeasible [2]. Consequently, log analysis and anomaly detection have been proposed as effective technologies to monitor the security status and therefore safeguard computer systems [3]–[5].

Log anomaly detection refers to identifying atypical behaviors that fall outside the expected patterns of normal system operations. Anomalies in logs indicate the presence of system failures and even malicious attacks. System operators leverage log anomaly detection methods to identify anomalies before further taking mitigation measures. Log anomaly detection methods utilize structured log data as inputs, which are typically generated by parsing raw logs. The objective of log anomaly detection is to accurately classify input logs as either normal or abnormal. Its core methodologies focus on analyzing discernible patterns within logs and distinguishing normal patterns from abnormal ones.

To accurately recognize patterns in log data, previous works propose to extract statistical features and then train machine learning (ML) models for anomaly detection [6]–[8]. The performance of these methods depends on the quality of extracted features. To this end, some works use deep learning (DL) approaches to automatically learn the sequential features in logs [4], [5], [9]. Some researchers propose to train RNN-based models for anomaly detection [4], [5]. Besides, with the rapid progress of DL, the Transformer architecture has taken overwhelming advantages in various fields and is leveraged for log-based anomaly detection [9], [10].

However, the current log anomaly detection methods cannot meet the security requirements of real-world complex computer systems. Firstly, the log data becomes more complex as modern computer systems evolve. This trend leads to the demand for developing more powerful anomaly detectors. Besides, existing methods train anomaly detectors on labeled datasets, which are difficult to acquire in real-world scenarios, as the amount of log data is extremely large and data labeling needs much labor work.

Since logs are essentially time series data, we design a Transformer-based model to acquire both learning capabilities and computing efficiency. We propose a novel training method following the pre-training and fine-tuning paradigm as in Large Language Models [11]. Specifically, to exploit unlabeled data, we pre-train a foundation model to learn the sequential features in logs with a novel self-supervised learning approach, i.e., masked log reconstruction. Then, we fine-tune it for the event prediction task to discern abnormal patterns and gain advanced

* Corresponding author
This work is supported by Key Area Research and Development Program of Guangdong Province under Grant No.2020B0101110003, NSFC Fund under Grant No.U20A20156, Grant No.62002342, and National Key Research and Development Program of China under Grant No.2020YFA0711400.

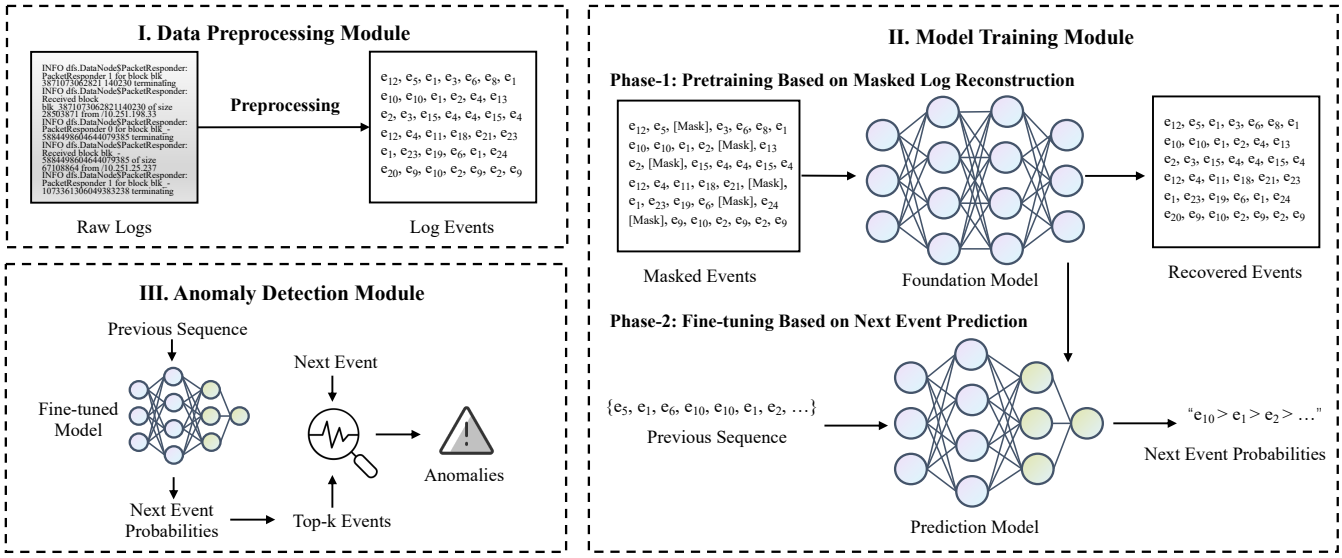

Fig. 1. The overview of our framework.

detection capabilities for complex log data. In this paper, we implement a log anomaly detection framework containing data processing, model training, and anomaly detection modules. The data processing module first casts raw logs into structured sequences. Then, the model training module trains the anomaly detector using our novel pre-training and fine-tuning approaches. Finally, in the anomaly detection module, the detector distinguishes anomalous patterns by comparing its predictions with ground-truth events.

We evaluate the performance of our framework along with six baseline models on two public datasets. Experimental results show that our framework outperforms these baseline models. More importantly, it is suitable for detecting anomalies in real-world scenarios.

To summarize, we have made the following contributions in this paper:

- We propose a novel framework for log anomaly detection. Different from existing works, our framework follows the pre-training and fine-tuning paradigm, which exploits unlabeled data and gains higher learning capabilities.
- We propose the masked log reconstruction approach to pre-train a Transformer-based foundation model and fine-tune it for the event prediction task to train the detector.
- We evaluate our framework on two public datasets and our framework outperforms the existing state-of-the-art methods. Also, we simulate real-world scenarios where unlabeled logs are hard to acquire. Our framework is the one that is still effective in such scenarios.

II. Related Work

As introduced above, previous works leverage both basic ML models and DL models for anomalous log detection. For ML-based methods, Xu *et al.* [6] propose to extract statistical features of events and parameters in raw logs. Then, they use the PCA algorithm to detect anomalies. Lin *et al.* [7] propose the LogCluster method, which clusters log sequences according to quantitative features and generates representative samples for anomaly detection. Such methods highly rely on the quality of extracted features, which are insufficient to cover all the complex sequential information and even may be ineffective over time. Therefore, these ML-based methods are difficult to deploy in production environments.

For DL-based methods, most works detect abnormal patterns in a way similar to processing natural language. DeepLog [4] first uses LSTM models to learn the contextual features in normal log sequences and detect abnormal patterns based on predictions. LogAnomaly *et al.* [5] extends DeepLog and proposes the Template2Vec approach to extract both sequential and quantitative features. In addition to LSTM, Transformer-based models are also introduced to the anomaly detection domain. Guo *et al.* [9] propose the LogBERT framework, which trains a BERT-like model as the anomaly detector by generating masked tokens in normal sequences.

Instead of adopting supervised learning methods, we introduce the pre-train and fine-tuning paradigm to the log analysis domain and propose a novel anomaly detection framework. We pre-train a foundation model to exploit unlabeled data, which is beyond the capabilities of existing methods. We fine-tune it for the event prediction task to discern outlier patterns.

III. Pre-trained Transformers for Log-based Anomaly Detection

A. Overview of our Framework

In this paper, we introduce the pre-training and fine-tuning paradigm to the log analysis domain and propose a novel log anomaly detection framework. We first pre-train a foundation model on large-scale unlabeled log data to learn the basic sequential features. Then, we fine-tune it on labeled logs to discern outlier patterns and gain anomaly detection capabilities. The trained detector identifies anomalous patterns

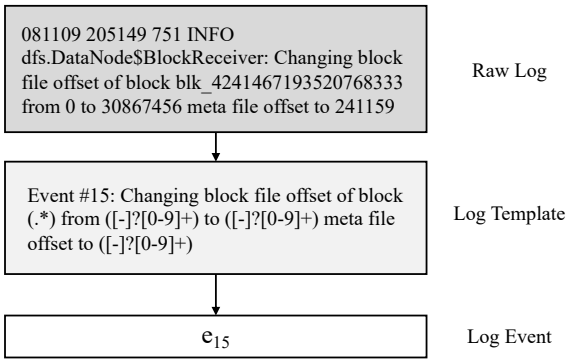

Fig. 2. An example of raw logs, log templates, and log events from the HDFS dataset.

by comparing its predictions with ground-truth events. Our framework exploits the sequential information within logs to achieve higher performance in real-world scenarios.

As shown in Figure 1, our framework includes three modules: (i) Data Preprocessing, (ii) Model Training, and (iii) Anomaly Detection. Specifically, raw logs are processed into event sequences in the first module. Then, the second module uses the extracted sequences to train the foundation model and then the prediction model with self-supervised learning. Finally, the trained prediction model detects outlier patterns in the third module. We give a detailed introduction to our framework as follows.

1) Data Preprocessing Module: Logs are formatted strings that record key information during system operation. Loggers typically populate pre-defined templates and print out logs. Therefore, massive raw logs can be processed into events according to log templates, which significantly reduces the amount of input data. We sort the extracted events by timestamp to generate event sequences. To illustrate the data preprocessing module, samples of raw logs, log templates, and extracted log events are shown in Figure 2. After that, these event sequences serve as input data for subsequent modules.

2) Model Training Module: According to previous works [4], [5], there are specific sequential patterns within logs that distinguish normal sequences from abnormal ones. The objective of log anomaly detection is to identify these patterns from time-series log data, which is highly similar to processing and analyzing natural language. In other words, we aim to understand the language that log data speaks. Therefore, we design our model training module referring to some state-of-the-art techniques in the NLP field.

Log event sequences are first encoded into vectors. Since the scale of log templates is relatively small in common systems, we select One-Hot encoding as the vectorization approach, which is simple yet sufficient for our task. Our model training module follows the pre-training and fine-tuning paradigm. Specifically, in the pre-training phase, we propose the masked log reconstruction-based technique to train a foundation model, which learns the contextual features in event sequences. Then, in the fine-tuning phase, we train the foundation model to make correct predictions on the next log events in normal sequences, which actually gives the model the ability to learn normal patterns. The fine-tuned prediction model is leveraged in the third module.

3) Anomaly Detection Module: We deploy the fine-tuned event prediction model as the log anomaly detection model, which continuously predicts the next events based on previous event sequences. Since the prediction model is trained to learn normal patterns, it detects anomalies by simply comparing the predictions with actual events.

B. Pre-training Based on Masked Log Reconstruction

In the pre-training phase, the foundation model learns the semantic features in logs. Transformers [10] have achieved dominant advantages in multiple fields, which are extremely suitable for processing time-series data and significantly outperform RNN models. Therefore, we leverage Transformer as the basis of our foundation model. It leverages multiple Transformer encoder layers to learn the sequential patterns and concatenates fully connected layers to generate outputs.

Inspired by BERT [12] and Masked Autoencoder [13], we pre-train the foundation model using input masking techniques, the core idea of which is to train models to learn key features by recovering masked inputs. In this paper, we introduce the input masking technique to the field of log analysis and propose a masked log reconstruction approach for pre-training. Our approach randomly masks part of the input events and trains the model to automatically recover the original sequences.

To illustrate our approach, supposing that an original sequence S_{origin} has L events, then we have $S_{origin} = \{S^1_{origin}, S^2_{origin}, S^3_{origin}, ..., S^L_{origin}\}$. To acquire the masked sequence, we randomly choose $\lfloor L * r \rfloor$ events with a masking ratio (r) and replace them with a masking token ($[mask]$). Assuming that the second event is masked, the masked sequence (S_{mask}) is formulated as $S_{mask} = \{S^1_{origin}, [mask], S^3_{origin}, ..., S^n_{origin}\}$. Given masked sequences, the foundation model is trained to accurately recover the masked sequences, i.e., to maximize the probabilities of outputting the corresponding original sequences. Therefore, the loss function is defined as the distance between the output sequence ($X_{recover}$) and the original sequence (X_{origin}). In this paper, we leverage the Mean-Squared Error (MSE) loss to measure the distance. Let X^j be the j^{st} value in the vectors, the loss function of our pre-training approach is formulated as

$$\begin{aligned} Loss_{pre-train} &= MSE\_Loss(X_{origin}, X_{recover}) \\ &= \frac{1}{L}\sum_{j=1}^{L}(X^j_{origin} - X^j_{recover})^2. \end{aligned} \tag{1}$$

To train the foundation model, we continuously update its parameters to minimize the loss value. After pre-training, the foundation model obtains the contextual information in the input log sequences. The trained foundation model is further fine-tuned in the next phase.

C. Fine-tuning Based on Next Event Prediction

As mentioned above, the foundation model learns semantic features of log data. Despite containing valuable information, it is insufficient for detecting anomalies. Therefore, we fine-tune the foundation model to a downstream task, i.e., next event prediction, which aims to predict the next events according to previous sequences. We train the model on labeled data so that it learns specific patterns distinguishing normal sequences from abnormal ones.

To accomplish the prediction task, we initially modify the structure of the trained foundation model by removing its output layers and freezing the parameters of the Transformer encoder layers since the semantic features are contained in these layers. Then, we concatenate a fully connected classifier to the modified foundation model, which aims to output the probabilities of the next events. Actually, other ML models that have prediction capabilities can act as the classifier, e.g., LSTM, 1D-CNN, etc. We fine-tune the prediction model by continuously optimizing the parameters of the concatenate classifier. Consequently, the fine-tuning phase requires much less training time than pre-training.

The modified model takes previous sequences as inputs and outputs the predictions of the next events, which are vectors containing the probabilities that different events occur next. To train the prediction model, we iteratively sample fixed-length sequences from labeled datasets as inputs and the next events as labels. Given previous sequences, the model aims to maximize the probabilities that target events are correctly predicted. Therefore, the loss function is defined as the difference between the output probabilities (Y_{output}) and the vectorized target label (Y_{true}). Using the Cross-Entropy (CE) loss to measure the difference, the loss function is formulated as

$$\begin{aligned} Loss_{fine-tune} &= CE\_Loss(Y_{output}, Y_{true}) \\ &= -\sum_{i=1}^{T} Y_{true}^{i} \log Y_{output}^{i}, \end{aligned} \tag{2}$$

where T is the number of log event types and Y^i is the probability value of the i^{st} event.

In the fine-tuning phase, all the training data are extracted from normal logs. After training, the prediction model learns normal sequential patterns and can accurately predict normal events. The fine-tuned prediction model is adopted for log anomaly detection.

D. Log Anomaly Detection

After fine-tuning, the prediction model learns the sequential patterns of normal logs. It is capable of predicting the next event according to normal previous sequences. Therefore, the behaviors of a functioning system can be precisely predicted by the fine-tuned model. In this way, outlier system operations that are beyond the expectations of the model are detected as anomalies. Inspired by this, we implement a prediction-based anomaly detection approach in our framework.

Specifically, given input sequences, the fine-tuned model outputs vectors containing the probabilities that log events occur next. As the fine-tuned prediction model makes accurate predictions on normal logs, the ground truth events are expected to have a higher value in the output probabilities, otherwise, anomalies are detected.

We also design an algorithm for online anomaly detection. Firstly, we iteratively extract fixed-length event sub-sequences as inputs and the next events as ground truth labels. Then, the extracted sub-sequences are fed into the fine-tuned prediction model to acquire output probabilities. We sort these probabilities in descending order and pick the top-k corresponding events as predictions. If a ground truth event does not appear in the predictions, an anomaly is detected and the whole sequence is labeled as abnormal.

IV. Evaluation

A. Evaluation Settings

Datasets. We evaluate our method on two public log datasets, including the HDFS dataset [14] and the BGL dataset [15]. The detailed information of these datasets is introduced as follows.

- HDFS Dataset. The Hadoop File System (HDFS) dataset contains 11,175,629 logs collected from a Hadoop cluster deployed on 200 Amazon EC2 nodes. These logs record 29 different types of events in the Hadoop system. The dataset is further processed and labeled. There are 575,061 blocks in the cluster, 16,838 of which are manually labeled as abnormal.
- BGL Dataset. The BGL dataset is collected from the BlueGene/L (BGL) supercomputer system at Lawrence Livermore National Labs (LLNL). The BGL dataset contains 4,747,963 logs, among which 348,460 are labeled as abnormal. These logs include 377 different types of log events in the BGL supercomputer.

In this paper, we evaluate the overall performance of our method on both HDFS and BGL datasets. Limited by the size of the BGL dataset, we only evaluate real-world scenarios on the HDFS dataset.

Baselines. In this paper, we compare our framework with six baseline methods. For ML-based methods, we evaluate PCA [14], Isolation Forest [8], and LogCluster [7] implemented in Loglizer [2]. For DL-based methods, we evaluate three state-of-the-art anomaly detectors, including DeepLog [4], LogAnomaly [5], and LogBERT [9].

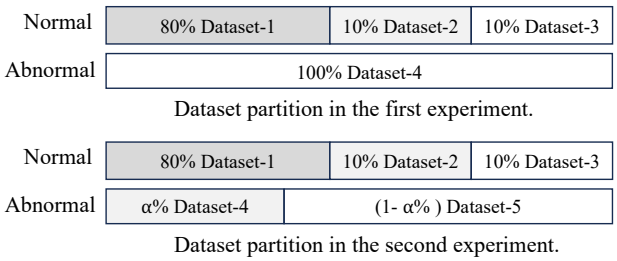

Fig. 3. Dataset partition methods in our experiments.

TABLE I
EVALUATION OF OVERALL PERFORMANCE.

	Datasets	HDFS				BGL			
	Metrics	Accuracy	Precision	Recall	F1 Score	Accuracy	Precision	Recall	F1 Score
Methods	PCA	1.76%	15.33%	100.00%	26.58%	50.25%	50.30%	99.80%	66.89%
	Isolation Forest	97.74%	95.98%	88.97%	92.34%	59.90%	100.00%	20.36%	33.83%
	LogCluster	90.26%	100.00%	36.44%	53.41%	81.69%	100.00%	63.64%	77.78%
	DeepLog	98.66%	97.62%	96.55%	97.08%	97.32%	99.03%	96.68%	97.84%
	LogAnomaly	98.97%	96.87%	98.74%	97.79%	96.50%	97.61%	96.79%	97.20%
	LogBERT	99.68%	99.03%	98.95%	98.99%	96.55%	99.06%	95.74%	97.37%
	Our Method	99.75%	99.50%	99.43%	**99.47%**	98.81%	99.28%	98.81%	**99.05%**

Implementation Details. We utilize regex to extract log events according to log templates and extract fixed-length sequences with $sequence\_length = 10$. Then, we leverage one-hot encoding to obtain vectorized representations. We initialize the foundation model with $hidden\_size = 128$. We set the number of Transformer encoder layers as 12 for the HDFS dataset and 4 for the BGL dataset. We randomly mask 10% of the input events to pre-train the foundation model. To perform prediction-based anomaly detection, considering the size of event templates, the k values are set as 3 and 39 for the HDFS and BGL datasets, respectively. For the baselines, we set $num\_layers = 2$ and $hidden\_size = 64$ for both DeepLog and LogAnomaly methods. We set $num\_layers = 4$ and $num\_headers = 4$ for LogBERT. We train these models for 500 epochs to achieve the best performance.

We run all the experiments on a server with Ubuntu 20.04, Intel Xeon 2.90 GHz CPU, 256G memory, and NVIDIA RTX 3090 GPU. We implement our method and the baseline models with Python 3.8 and PyTorch 1.11.

Evaluation Metrics. As the testing data is unbalanced in our datasets, we count the TP, FP, TN, and FN values and calculate four metrics to evaluate our method and the baseline models, including $Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$, $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$, and $F1\ Score = 2 * \frac{Precision*Recall}{Precision+Recall}$.

B. Evaluation of Overall Performance

In the first experiment, we compare the overall performance of our framework with baselines on the HDFS and BGL datasets. These datasets are split as shown in Figure 3. For our framework, we pre-train the foundation model on Dataset 1 and fine-tune it on Dataset 2. For the baselines, we train them on a dataset combining Dataset 1 and Dataset 2. We test all the methods on Dataset 3 and Dataset 4.

Experimental results are shown in Table I. Overall, our framework achieves the best performance among all the methods, which indicates that it learns semantic features in normal log data and effectively detects outlier log patterns. Although ML-based methods achieve high performance in specific metrics (e.g., PCA achieves 100% Recall on the HDFS dataset), their overall performance is much lower than DL-based methods, which proves that such methods have lower learning abilities than DL models. On the HDFS dataset, our method achieves 99.75% accuracy, 99.50% precision, 99.43% recall, and 99.47% F1 score, all of which outperform the DL-based baseline models. Besides, on the BGL dataset, three DL-based baseline models achieve comparable performance. Our method achieves 98.81% accuracy, 99.28% precision, 98.81% recall, and 99.05% F1 score, significantly outperforming the DL-based baselines. The experimental results indicate that our method detects more anomalies while triggering fewer false alarms, which is crucial for deploying anomaly detectors in production environments.

It is also shown that the Transformer architecture and our novel training methods play a significant role. Among the DL-based detectors, both our method and LogBERT utilize the Transformer architecture to extract and learn the sequential features within log data, while DeepLog and LogAnomaly use the LSTM model. Evaluation results show that these Transformer-based methods outperform the LSTM-based ones, which proves the superiority of the Transformer-based architecture. Moreover, our method performs better than the LogBERT model on both HDFS and BGL datasets, which indicates that following the pre-training and fine-tuning paradigm, our novel training method makes the model learn more key features and better distinguish unexpected patterns.

To conclude, in the first experiment, we train and evaluate our method and six baseline models on two public datasets where all the data are labeled, which conducts an overall evaluation of all these methods and provides unified metrics of their performance. Experimental results show that our framework achieves the best performance among these methods. In the following experiment, we will further prove the superiority of our framework in real-world scenarios.

C. Evaluation of Real-world Scenarios

As described above, existing works typically train their anomaly detection models on normal log data and test them on both normal and abnormal data. They assume that all

TABLE II
EVALUATION OF REAL-WORLD SCENARIOS.

Ratio	Methods	Accuracy	Precision	Recall	F1 Score
20%	DeepLog	93.19%	95.82%	67.92%	79.49%
	LogAnomaly	87.97%	86.20%	45.39%	59.46%
	LogBERT	94.74%	95.43%	64.19%	76.75%
	Our Method	99.76%	99.35%	99.44%	**99.39%**
40%	DeepLog	93.63%	93.74%	62.58%	75.05%
	LogAnomaly	90.29%	82.16%	46.78%	59.63%
	LogBERT	94.57%	93.25%	53.95%	68.35%
	Our Method	99.79%	99.20%	99.43%	**99.31%**
60%	DeepLog	95.00%	90.85%	59.56%	71.95%
	LogAnomaly	91.08%	70.87%	29.14%	41.30%
	LogBERT	96.44%	90.27%	56.95%	69.84%
	Our Method	99.34%	98.74%	95.12%	**96.89%**

the log data in the dataset is labeled. However, labeled log datasets are extremely hard to obtain as manually labeling logs requires a heavy workload. Therefore, the amount of unlabeled log data is significantly larger than labeled ones in real-world scenarios. We believe that it is essential to evaluate anomaly detectors under such circumstances. In the subsequent experiment, we split the HDFS dataset differently to simulate real-world scenarios. As shown in Figure 3, abnormal data is divided into two parts, where Dataset 4 contains $\alpha\%$ of it and Dataset 5 contains the rest. For our framework, we first pre-train the foundation model on an unlabeled dataset combining Dataset 1 and Dataset 4 and then fine-tune it on Dataset 2. For the baselines, we train them on a dataset combining Dataset 1, Dataset 2, and Dataset 4 to obtain fair evaluation results. All the methods are tested on Dataset 3 and Dataset 5.

We select the value of α as 20, 40, and 60 to construct our datasets and run the experiments on these datasets separately. As shown in Table II, the evaluation metrics prove that our framework remarkably outperforms the baselines. More importantly, the performance of baseline models drops significantly compared with the first experiment, while our framework achieves similar performance, which makes our method the only one that can adapt to real-world scenarios.

As trained on unlabeled datasets, the baselines learn the sequential features within normal and abnormal logs, making these patterns more indistinct and thus resulting in significant degradation in performance. Unlike them, our framework first pre-trains the foundation model to exploit unlabeled data and then fine-tunes it on labeled data to acquire detection capabilities. Consequently, our framework performs well in real-world scenarios. Besides, it is also shown that the performance of our framework declines slightly as α increases. With more abnormal data combined in the pre-training phase, the foundation model actually learns more abnormal features. Given the same fine-tuning dataset, it is more difficult for the fine-tuned model to distinguish between normal and abnormal patterns. Despite the slight degradation in performance, our framework achieves acceptable evaluation results.

To summarize, in the second experiment, we prove that our framework is suitable for real-world scenarios. Among all the DL methods, our framework is the only one to achieve high performance in such scenarios.

V. CONCLUSION

We propose a novel framework for log anomaly detection in this work. Our framework follows the pre-training and fine-tuning paradigm to exploit log data and acquire better learning capabilities. We propose the masked log reconstruction approach to pre-train a Transformer-based foundation model and fine-tune it as the anomaly detector. Evaluation results indicate that our framework outperforms the existing state-of-the-art methods and is extremely suitable for real-world scenarios.

REFERENCES

[1] A. J. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," *Commun. ACM*, vol. 55, no. 2, pp. 55–61, 2012.

[2] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE ISSRE*, Ottawa, ON, Canada, Oct. 2016, pp. 207–218.

[3] M. Du and F. Li, "Spell: Online streaming parsing of large unstructured system logs," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 11, pp. 2213–2227, 2019.

[4] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM CCS*, Dallas, TX, USA, Oct. 2017, pp. 1285–1298.

[5] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. IJCAI*, Macao, China, Aug. 2019, pp. 4739–4745.

[6] W. Xu, L. Huang, A. Fox, D. A. Patterson, and M. I. Jordan, "Online system problem detection by mining patterns of console logs," in *Proc. IEEE ICDM*, Miami, FL, USA, Dec. 2009, pp. 588–597.

[7] Q. Lin, H. Zhang, J. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. IEEE ICSE*, L. K. Dillon, W. Visser, and L. A. Williams, Eds., Austin, TX, USA, May 2016, pp. 102–111.

[8] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," in *Proc. IEEE ICDM*, Pisa, Italy, Dec. 2008, pp. 413–422.

[9] H. Guo, S. Yuan, and X. Wu, "Logbert: Log anomaly detection via BERT," in *Proc. IJCNN*, Shenzhen, China, Jul. 2021, pp. 1–8.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NIPS*, Long Beach, CA, USA, Dec. 2017, pp. 5998–6008.

[11] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proc. NIPS*, Nov. 2022.

[12] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186.

[13] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. B. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF CVPR*, New Orleans, LA, USA, Jun. 2022, pp. 15 979–15 988.

[14] W. Xu, L. Huang, A. Fox, D. A. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ICML*, Haifa, Israel, Jun. 2010, pp. 37–46.

[15] A. J. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. IEEE/IFIP DSN*, Edinburgh, UK, Jun. 2007, pp. 575–584.